

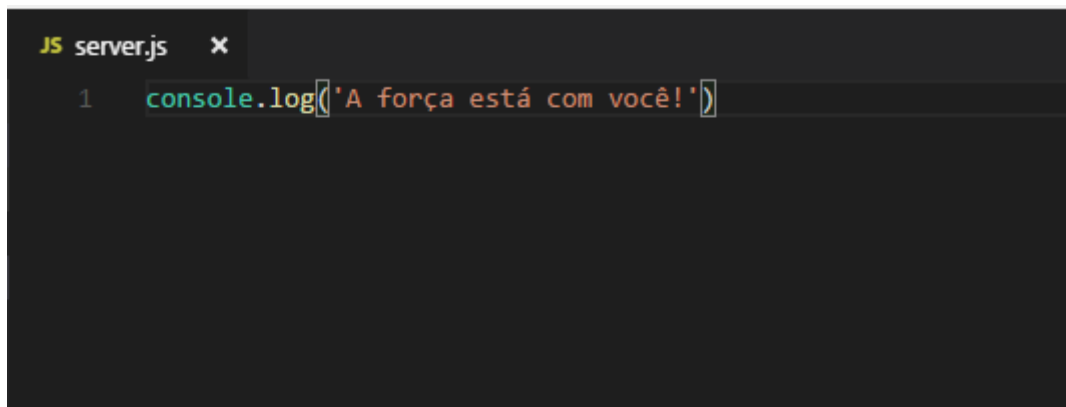
Início do projeto - MongoDB

Vamos iniciar nosso projeto, para tal, vamos abrir nosso terminal e criar uma pasta onde alocaremos nossos arquivos, depois entramos na pasta e via CLI abrimos o nosso editor de texto (escolha o seu favorito):

Voltando ao terminal, vamos rodar o comando `npm init`. Esse comando cria o `package.json`, arquivo que irá nos ajudar a gerenciar as dependências que instalaremos mais tarde nesse tutorial.

`npm init`

Agora iremos criar nosso arquivo principal, `server.js` e dentro dele escrevemos o seguinte código:

A screenshot of a code editor with a dark background. The top bar shows 'JS server.js' and a close button. The main area contains a single line of code: `1 console.log('A força está com você!')`.

```
JS server.js x
1 console.log('A força está com você!')
```

Você deverá ver a informação abaixo no seu terminal:

A screenshot of a terminal window with a dark background. It shows the command `λ node server.js` being entered, followed by the output `A força está com você!`.

```
λ node server.js
A força está com você!
```

Usando Express

Nós devemos instalar o Express antes de poder usá-lo em nosso app. Instalar o Express é bem simples, tudo que precisamos é rodar um código via CLI usando o gerenciador de pacotes NPM. Rode o código abaixo:

```
npm install express --save :
```

Feito isso você verá que o npm salvou o Express como uma dependência no `package.json`.

```
9   "author": "",
10  "license": "ISC",
11  "dependencies": {
12    "express": "^4.16.3"
13  }
14 }
```

Agora vamos usar o express no nosso arquivo `server.js`. Para isso vamos excluir o `console.log()` que tínhamos e colocar o código abaixo:

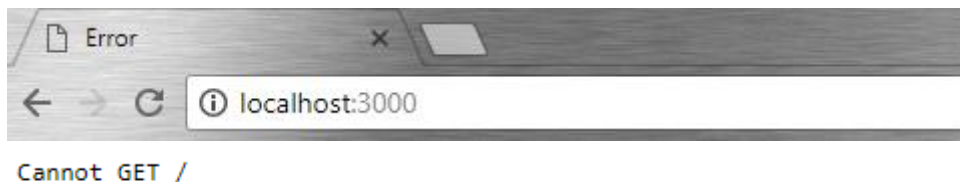
```
JS server.js x
1  const express = require('express')
2  const app = express()
```

Precisamos fazer com que nosso servidor e o navegador possam se comunicar. Podemos fazer isso com a ajuda do método `listen` fornecido pelo Express:

```
JS server.js x
1  const express = require('express')
2  const app = express()
3
4  app.listen(3000, function() {
5    console.log('server running on port 3000')
6  })
```

Método para iniciar a comunicação com o servidor

Rodando no terminal o comando `node server.js`, vamos acessar o caminho `localhost:3000` em nosso navegador. Você verá a mensagem a seguir:



Nosso navegador e servidor já estão se comunicando

Iniciando nosso CRUD – READ

Iremos começar explicando a operação READ, pois é bem simples de entender o nosso cenário atual. Esta operação é executada pelos navegadores sempre que você visita uma página da web. Ao ser iniciado, os navegadores enviam uma solicitação **GET** ao servidor para executar uma operação de **leitura**. A razão pela qual vemos o erro “cannot get /” é porque ainda temos que enviar alguma informação de volta para o navegador, que esteja vindo do nosso servidor.

Usando Express, essa solicitação é feita com o método GET e pode ser escrito da seguinte maneira:

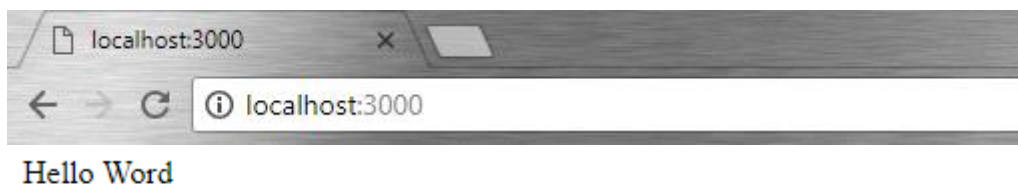
```
4 app.get('/', (req, res) => {  
5   |   res.send('Hello World')  
6   |  
7   | })
```

Note que incluímos em nosso arquivo server.js o trecho de código acima, veja que nele estamos passando o caminho no qual o navegador está acessando que em nosso caso é localhost:3000/, e para isso o primeiro argumento é “/”. O segundo argumento é uma função callback que informa ao servidor o que fazer quando o caminho é correspondido. Esse callback leva dois argumentos, um objeto de solicitação e outro de resposta (req e res, respectivamente). A fim de

testarmos se está indo tudo ok, enviaremos somente a frase 'Hello Word' com o método `send` para nos trazer o objeto de resposta. Note que estou usando o padrão ES6, e por isso usei diretamente uma arrow function.

Agora vamos reiniciar nosso servidor, basta fazer o seguinte:

- Pare o servidor atual pressionando `CTRL + C` no seu terminal.
- Rode novamente o comando `node server.js`.
- Em seguida, acesse `localhost:3000` no seu navegador. Você deve estar vendo uma string que diz: Hello World.



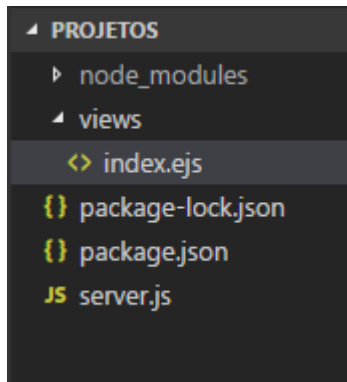
Vamos agora iniciar a parte visual do nosso projeto, para isso iremos utilizar um template engine chamado EJS (Embedded Javascript), ele é bem simples de usar principalmente para quem tem familiaridade com HTML e Javascript. Para instalar o ejs, basta digitar no seu terminal:

```
npm install ejs -save
```

Precisamos configurar nossa view engine no Express, basta incluir o trecho de código abaixo.

```
3
4  app.set('view engine', 'ejs')
5
```

Feito isso, poderemos gerar código HTML que será renderizado em nosso navegador. Vamos então criar nosso arquivo `index.ejs` dentro de uma pasta chamada 'views', veja como ficará nossa estrutura:



```
<!DOCTYPE
html>

<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Meu CRUD em NodeJS</title>
</head>
<body>

  <h2>Registrar</h2>
  <form action="/show" method="POST">
    <input type="text" placeholder="Nome" name="name">
    <input type="text" placeholder="Sobrenome" name="surname">
    <button type="submit">Submit</button>
  </form>

</body>
</html>
```

Agora precisamos setar nosso arquivo para que ele seja enviado pro servidor, e ser renderizado no navegador. Para isso precisamos fazer uma alteração no trecho de código `app.get()`. Veja abaixo a alteração:

```
4 app.set('view engine', 'ejs')
5
6 app.get('/', (req, res) => {
7   |   res.render('index.ejs')
8   | })
9
```

Certo, agora precisamos reiniciar nosso servidor, para visualizar a alteração em nosso navegador. Vá ao seu terminal e pressione `CTRL + C`, em seguida digite novamente `node server.js`. Agora atualizando a página no seu navegador você terá esse resultado:



Registrar

Nome	Sobrenome	Submit
-----------------------------------	--	---------------------------------------

Conteúdo renderizado pelo navegador

CRUD – CREATE

A operação **CREATE** é executada apenas pelo navegador se uma solicitação **POST** for enviada ao servidor. Essa solicitação POST pode ser acionada com JavaScript ou por meio de um elemento `<form>`.

Vamos descobrir como usar um elemento `<form>` para criar novas entradas para o nosso projeto nesta parte do tutorial. Para fazer isso, iremos utilizar o form já criado em nosso arquivo `index.ejs`.

Você precisa ter três coisas neste elemento de formulário:

- Um atributo `action`.

- Um atributo `method` .
- E um atributo `name` para todo elemento `<input>` do formulário.

```
8
9  <h2>Registrar</h2>
10  <form action="/show" method="POST">
11    <input type="text" placeholder="Nome" name="name">
12    <input type="text" placeholder="Sobrenome" name="surname">
13    <button type="submit">Submit</button>
14  </form>
15
```

Parte do nosso arquivo `index.ejs`

O atributo `action` informa ao navegador para onde redirecionar nosso app Express. Nesse caso estamos sendo direcionados para `/show` . O atributo `method` informa ao navegador qual solicitação enviar, nesse caso é uma solicitação do tipo POST.

Em nosso servidor, podemos processar essa solicitação POST com um método `post` fornecido pelo Express que leva os mesmos argumentos do método GET:

```
10  app.post('/show', (req, res) => {
11    |    console.log('Hello again..')
12  })
13
```

Após isso, salve as alterações. Note que em seu terminal o nodemon irá reiniciar o servidor automaticamente, então vá no seu navegador e preencha o formulário e clique em enviar. Você poderá ver em seu terminal a seguinte mensagem:

```
[nodemon] 1.17.5
[nodemon] to restart at any time, enter `rs`
[nodemon] watching: *.*
[nodemon] starting `node server.js`
server running on port 3000
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
server running on port 3000
Hello again..
```

Servidor reiniciado automaticamente + mensagem no console

Ótimo, sabemos que o Express está lidando direitinho com o formulário. Mas pergunta que fica é: como obtemos os valores de entrada com o Express?

Acontece que o Express não lida com a leitura de dados do elemento `<form>` por conta própria. Temos que adicionar outro pacote chamado **body-parser** para conseguir essa funcionalidade, pare o servidor e digite em seu terminal:

```
npm install body-parser -save
```

O Express nos permite adicionar middleware como body-parser ao nosso aplicativo com o método `use`. Você ouvirá muito o termo *middleware* ao lidar com o Express, eles são basicamente plugins que alteram o objeto request ou response antes de serem manipulados pelo nosso aplicativo.

```
JS server.js x <> index.ejs {} package.json
1  const express = require('express')
2  const bodyParser = require('body-parser')
3  const app = express()
4
5  app.use(bodyParser.urlencoded({ extended: true }))
6
```

Inclua bodyParser em seu arquivo server.js

O método `urlencoded` dentro de body-parser diz ao body-parser para extrair dados do elemento `<form>` e adicioná-los à propriedade `body` no objeto `request`. Faça o teste, dê um `console.log` e preencha o formulário, você deverá ver tudo no campo de formulário dentro do objeto:


```

13   app.post('/show', (req, res) => {
14     |   console.log(req.body)
15   })
16

```

Você verá em seu terminal algo parecido com isto:

```

PS C:\Users\Flavio Mota\OneDrive - Etec Centro Paula Souza\POAD\2 SEMESTRE\ProjetoNodeMongoDB\Sistema> nodemon server
[nodemon] 2.0.4
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node server.js`
Server rodando na porta 3000
{ name: 'Flavio', surname: 'Mota' }

```

MongoDB

Para usarmos o MongoDB como banco de dados, nós precisamos instalá-lo e podemos fazer isso através do npm conforme abaixo.

```
npm install mongodb --save
```

Uma vez instalado, podemos nos conectar ao MongoDB através do método de conexão do MongoClient.connect, conforme mostrado no código abaixo:

```

const express = require('express')
const app = express()
const bodyParser = require('body-parser')
app.set('view engine', 'ejs')
app.use(bodyParser.urlencoded({extended: true}))
const MongoClient = require('mongodb').MongoClient

const caminho = "mongodb://localhost/Escola"

MongoClient.connect(caminho, {useNewUrlParser: true, useUnifiedTopology: true}, (err, client) => {
  |   db = client.db('Escola')
  | })

```

Salvando nossos dados no banco

Iremos criar a coleção “data”, que irá armazenar nossos dados, apenas colocando-a entre aspas ao chamar o método db.collection() do MongoDB. Ao criar a coleção, também podemos salvar nossa primeira entrada no MongoDB com o método save simultaneamente.

```
MongoClient.connect(caminho,{useNewUrlParser: true, useUnifiedTopology: true},{err, client} => {  
  db = client.db('Escola')  
})  
  
app.get('/',(req,res)=>{  
  res.render('index.ejs')  
})  
app.post('/show',(req,res)=>{  
  db.collection('data').save(req.body, (err,result)=>{  
    console.log('Dados salvo com sucesso!!')  
    res.redirect('/')  
  })  
})  
  
app.listen(3000,function() {  
  console.log('Server rodando na porta 3000')  
})
```

Quando terminarmos de salvar, precisamos redirecionar o usuário para algum lugar (senão o usuário ficará esperando para sempre até que nosso servidor seja alterado). Nesse caso, vamos redirecioná-los de volta para */*, o que faz com que os navegadores sejam recarregados.

CRUD — READ

Mostrando o conteúdo de nossa coleção.

Temos que fazer algumas coisas para mostrar o conteúdo armazenado em nosso banco de dados para o nosso usuário. Precisamos:

- Pegar o conteúdo no banco de dados.
- Usar nossa template engine (**EJS**) para exibir esse conteúdo.

Podemos obter o conteúdo do nosso BD usando o método `find()` disponível no método `collection`.

```

MongoClient.connect(caminho,{useNewUrlParser: true, useUnifiedTopology: true},{err, client) => {
  db = client.db('Escola')
})

app.get('/',(req,res)=>{
  db.collection('data').find().toArray((erro,resultado)=>{
    console.log(resultado)
  })
  res.render('index.ejs')
})

```

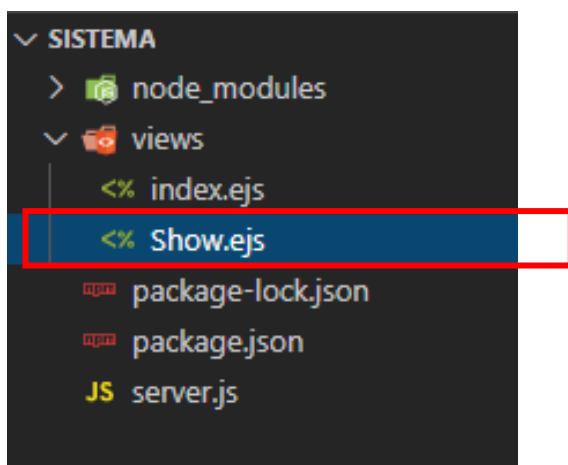
Após incluir o trecho acima, vá até seu navegador e preencha o <form> e clique em submit . Você verá o resultado em seu terminal.

```

PS C:\Users\Flavio Mota\OneDrive - Etec Centro Paula Souza\POAD\2 SEMESTRE\ProjetoNodeMongoDB\Sistema> nodemon server
[nodemon] 2.0.4
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node server.js`
Server rodando na porta 3000
[
  { _id: 5f7297231d02b94af44e15b1, name: 'Rochele', surname: 'Mota' },
  { _id: 5f7297761d02b94af44e15b2, name: 'Joao', surname: 'Mota' },
  { _id: 5f72981efe31104b28173b91, name: 'Flavio', surname: 'Mota' },
  { _id: 5f729893fe31104b28173b92, name: 'Iza', surname: 'Mota' },
  { _id: 5f729901a9c7544370e5c861, name: 'Jesus', surname: 'Mota' },
  { _id: 5f776d789dc76431f8e337df, name: 'Juciara', surname: 'Mota' }
]

```

Ótimo! Você está vendo o conteúdo preenchido (o mesmo que foi salvo no banco de dados). Concluímos a primeira parte, que é obter dados do servidor MongoDB. A próxima parte será renderizar esse conteúdo usando nossa template engine (EJS). Bom, para isso crie um novo arquivo dentro da pasta 'views' conforme abaixo.



Agora abra-o e crie uma tabela que mostre o conteúdo que enviamos pelo formulário.

```

<!DOCTYPE
html>

<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My CRUD in NodeJS</title>
</head>
<body>

<h2>Registros</h2>
  <table border="1">
    <thead>
      <tr>
        <td>Nome</td>
        <td>Sobrenome</td>
      </tr>
    </thead>
    <tbody>
      <% data.forEach(function(details) { %>
        <tr>
          <td><%= details.name %></td>
          <td><%= details.surname %></td>
        </tr>
      <% }) %>
    </tbody>
    <button><a href="/">Voltar</a></button>
  </body>
</html>

```

Note que na linha 18 nós estamos mostrando nossa variável `data` combinada com o método `.forEach()` onde irá percorrer o conteúdo da collection em nosso banco de dados, e criamos então a `function(details)` que nas linhas 20 e 21, irá mostrar o conteúdo que estamos chamando com `<%= details.name%>` e `<%= details.surname%>` respectivamente.

Em seguida vamos fazer uma pequena alteração no nosso `server.js`. Vamos deixá-lo conforme abaixo.

```

const express = require('express')
const app = express()
const bodyParser = require('body-parser')
app.set('view engine', 'ejs')
app.use(bodyParser.urlencoded({extended: true}))
const MongoClient = require('mongodb').MongoClient

const caminho = "mongodb://localhost/Escola"

MongoClient.connect(caminho, {useNewUrlParser: true, useUnifiedTopology: true}, (err, client) => {

    db = client.db('Escola') // Buscando no servidor MongoDB o banco Escola

})

app.get('/', (req, res) => {

    res.render('index.ejs')
})

app.get('/show', (req, res) => {
    db.collection('data').find().toArray((err, results) => {

        res.render('show.ejs', { data: results })

    })
})

app.post('/show', (req, res) => {
    db.collection('data').save(req.body, (err, result) => {
        res.redirect('/show')
    })
})

app.listen(3000, function() {
    console.log('Server rodando na porta 3000')
})

```

Volte ao navegador e preencha o formulário, você será redirecionado para a página `localhost:3000/show`, e verá o conteúdo abaixo (provavelmente seu conteúdo será diferente).

Registros

[Voltar](#)

Nome	Sobrenome
Rochele	Mota
João	Pereira
Flavio	Mota
Jesus	Mota
Antonia	Mota
Raquel	Mota
Joice	Mucci
Carlos	Pereira

Referencia : <https://www.luiztools.com.br/>